

Code Explanation: Phase 4 - Model Optimization & Training

Project: NeuroVision (Brain Tumor Classification) **Objective:** To configure the model's learning process and actively train it on our dataset, applying concepts from the Unit II (Optimizers) and Unit V Lab (Epochs, Batch Size, and Validation) syllabus.

1. Model Compilation

Before a model can start learning, it must be compiled. This step defines *how* the model will learn and measure its own mistakes.

- `model.compile(...)`: This function finalizes the model setup and prepares it for the training phase.
- `optimizer='adam'`: We assign the Adam (Adaptive Moment Estimation) optimizer. This is a highly efficient, popular algorithm that calculates how much to adjust the neural network's weights to minimize the loss and reduce the cost function.
- `loss='binary_crossentropy'`: This defines the specific mathematical function used to calculate the model's error (loss). Because we are solving a binary classification problem with only two outcomes (Tumor or Normal), "binary crossentropy" is the standard loss function used to measure the difference between the predicted probabilities and the actual labels.
- `metrics=['accuracy']`: This tells the model that we want to track its accuracy (the percentage of correct predictions) as the primary performance metric during the training process.

2. Model Training (Fitting)

This is where the actual machine learning happens. The model looks at the data, makes guesses, checks its errors, and updates its parameters.

- `history = model.fit(...)`: The `.fit()` command starts the training loop. We assign the output to a variable named `history` so we can save the logs of the training process (like how the accuracy improved over time) for later visualization.
- `X_train, y_train`: These are the training images and their corresponding correct labels that we prepared earlier. The model will study these to learn the patterns of a tumor.
- `epochs=20`: An "epoch" represents one complete forward and backward pass through the entire training dataset. Setting this to 20 means the model will study the full dataset 20 separate times to converge on the best mathematical weights.
- `validation_data=(X_test, y_test)`: After every single epoch, the model will test itself on this unseen testing data. By tracking both the training loss and the

validation loss, we can ensure the model is genuinely learning and not just memorizing the training data (overfitting).

- `batch_size=16`: Instead of looking at all 202 training images at once (which requires massive RAM) or one at a time (which is slow), the model will process the images in small batches of 16. It will update its weights after every batch, acting as a form of Stochastic/Mini-Batch Gradient Descent.